

МИНИСТЕРСТВО НАУКИ И ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФГБОУ ВО «НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»

Кафедра вычислительной техники

Курсовая работа

Исследование точности алгоритма и программы определения
направления на источник импульсных сейсмических сигналов на основе
расчета КНД в зависимости от количества датчиков.

по дисциплине: «Мониторинговые системы и сети»

Выполнили:

С

И

№ ____ » ____ 20 ____ г.

И

Р

Н

Б

Ы

Е

Г

И

.

Хайретдинов М.С.

А

М

М

Проверил:

« ____ » ____ 20 ____ г.

(подпись)

Новосибирск 2025

1 Введение

Данная работа посвящена исследованию точности алгоритма определения направления на источник импульсных сейсмических сигналов на основе расчета кинематического наклона волнового фронта (КНД) в зависимости от количества используемых датчиков. Исследование проводится на реальных данных промышленных полигонных взрывов в частотном диапазоне 1-20 Гц.

2 Цели работы

- ознакомление и практическое овладение методом обработки;
- ознакомление со средствами графического представления результатов обработки;
- ознакомление с организацией массивов представления данных и их форматами.
- ознакомление с особенностями получения экспериментального материала и его описание;
- сравнительный анализ точности работы алгоритма от количества используемых в линейке датчиков (2,3,4,5 точек регистрации)

Теоретическая часть

Определение направления на источник импульсных сейсмических сигналов является важной задачей в сейсморазведке и мониторинге природных и техногенных процессов. В основе рассматриваемого метода лежит анализ кинематики волнового фронта (КНД - кинематический наклон волнового фронта), который позволяет оценить направление прихода волны по данным с массива сейсмических датчиков.

Физической основой метода является анализ временных задержек прихода волны на различные датчики сейсмической антенны. При распространении сейсмической волны от источника фронт волны достигает датчиков,

расположенных на некотором расстоянии друг от друга, в разные моменты времени. Эти временные задержки содержат информацию о направлении распространения волны и могут быть использованы для определения азимута на источник.

Математически задача сводится к решению системы уравнений, связывающих координаты датчиков, времена прихода сигнала и предполагаемое положение источника. Для линейной антенны датчиков, расположенных вдоль оси X, основное уравнение имеет вид:

$$\Delta t_{ij} = (x_j - x_i) * \frac{\sin(\theta)}{v}$$

t_{ij} - разность времен прихода сигнала на i-й и j-й датчики

x_i и x_j - координаты датчиков

θ - угол между направлением на источник и нормалью к антенне

скорость распространения волны

При увеличении количества датчиков повышается точность определения направления за счет:

- увеличения количества независимых измерений временных задержек;
- возможности применения статистических методов обработки;
- уменьшения влияния погрешностей отдельных измерений.

Точность метода существенно зависит от следующих факторов:

- частотного состава;
- отношения сигнал/шум;
- точности определения времен прихода;
- геометрии расположения датчиков;
- количества используемых датчиков.

Теоретически, для линейной антенны минимальное количество датчиков, необходимое для оценки направления, равно двум. Однако на практике использование большего числа датчиков позволяет существенно повысить точность за счет усреднения результатов и применения методов оптимальной фильтрации. При этом существует нелинейная зависимость между количеством датчиков и достигаемой точностью - первоначальное увеличение числа датчиков дает значительный выигрыш в точности, который затем постепенно уменьшается.

Важным аспектом является обработка реальных сигналов, которые в отличие от модельных, содержат шумы, многолучевость и другие искажения. Это требует применения специальных методов обработки, таких как полосовая фильтрация в указанном частотном диапазоне, анализ огибающей сигнала и корреляционные методы определения времен прихода.

Описание данных

Каждая запись представлена в виде отдельного файла PC. Файлы состоят из заголовка и последовательно расположенных отсчетов. Формат заголовка приведен в таблице 2.1.

Таблица 2.1 – Формат заголовка файла

Смещ.	Длина	Тип и имя	Значение	Комментарий
				Идентификатор формата
				Номер версии
				Номер подверсии
				Длина заголовка в байтах
				Дата и примерное время начала в секундах с 1 Января 1970 г.

				Длительность сеанса в секундах
				Коэффициент пересчета в физическую величину
				Год начала сеанса
				Месяц начала сеанса
				День начала сеанса
				Час начала сеанса
				Минута начала сеанса
				Секунда начала сеанса
				Поправка в микросекундах к времени начала сеанса
				Частота дискретизации в Гц
				Число отсчетов в трассе
				Длина и тип отсчета: 0002h - int - знаковое 16 бит 0102h - unsigned – беззнаковое 0004h - long - двойное 32 бит 1004h - float - плавающая точка 1008h - double – двойное
				Номер трассы в исходном файле
				Зарезервировано, д.б. ноль

Формат самого файла:

Заголовок (N байт)
Отсчёт 1 (M байт)
Отсчёт 2 (M байт)
...
Отсчёт K (M байт)

3.2 Средства Python

В данном исследовании был применён комплекс специализированных инструментов, охватывающих все этапы работы с данными — от их загрузки до визуализации результатов.

Основой для работы с сейсмическими записями стала библиотека ObsPy, представляющая собой специализированный инструментарий для сейсмологии. Её ключевое преимущество — поддержка всех распространённых форматов хранения сейсмических данных, включая SEED, MiniSEED и SAC.

Библиотека ObsPy не только обеспечивает удобный доступ к данным, но и предоставляет встроенные методы для их обработки, такие как фильтрация, деконволюция и расчёт спектральных характеристик. Особенно важной возможностью является работа с метаданными станций, что критически важно для корректной интерпретации результатов.

Для проведения численных расчетов и обработки сигналов были использованы библиотеки NumPy. NumPy является фундаментальным инструментом для научных вычислений в Python и обеспечивает эффективную работу с многомерными массивами данных.

Визуализация результатов была выполнена с использованием Matplotlib — наиболее распространенной библиотеки для создания графиков в научной среде. Ее гибкость позволяет создавать как стандартные графики временных рядов, так и более сложные представления данных, такие как спектрограммы и полярные диаграммы направленности.

Практическая часть

Выполнение работы

В ходе выполнения курсовой работы, был реализован программный код на языке Python. Данный код представляет собой программную реализацию исследования точности алгоритма определения направления на источник сигнала (DOA - Direction of Arrival) и расчета коэффициента направленного действия (КНД) в зависимости от количества используемых датчиков.

Разработка предназначена для анализа сейсмических или акустических сигналов, зарегистрированных системой датчиков в полевых условиях.

Основной смысл программы заключается в следующем: система из нескольких датчиков, расположенных линейно с фиксированным расстоянием D между ними, регистрирует приходящий сигнал. Алгоритм анализирует временные задержки между сигналами на разных датчиках, чтобы определить направление на источник. Параллельно вычисляется коэффициент направленного действия, характеризующий способность системы усиливать сигнал в определенном направлении.

Код состоит из нескольких ключевых функций:

функция `parse_header` читает и интерпретирует заголовки файлов с данными, извлекая частоту дискретизации (fs) и количество точек данных

функция `read_channels` загружает данные со всех каналов (датчиков), приводит их к единому формату и формирует общую матрицу данных, где каждая колонка соответствует одному датчику.

функция `estimate_doa` является сердцем алгоритма определения направления. Для двух датчиков используется простое соотношение между задержкой и углом, для большего количества - метод линейной регрессии.

функции `calculate_directivity_pattern` и `calculate_directivity` рассчитывают диаграмму направленности и коэффициент направленного действия системы соответственно, используя теоретические модели для линейных антенных решеток.

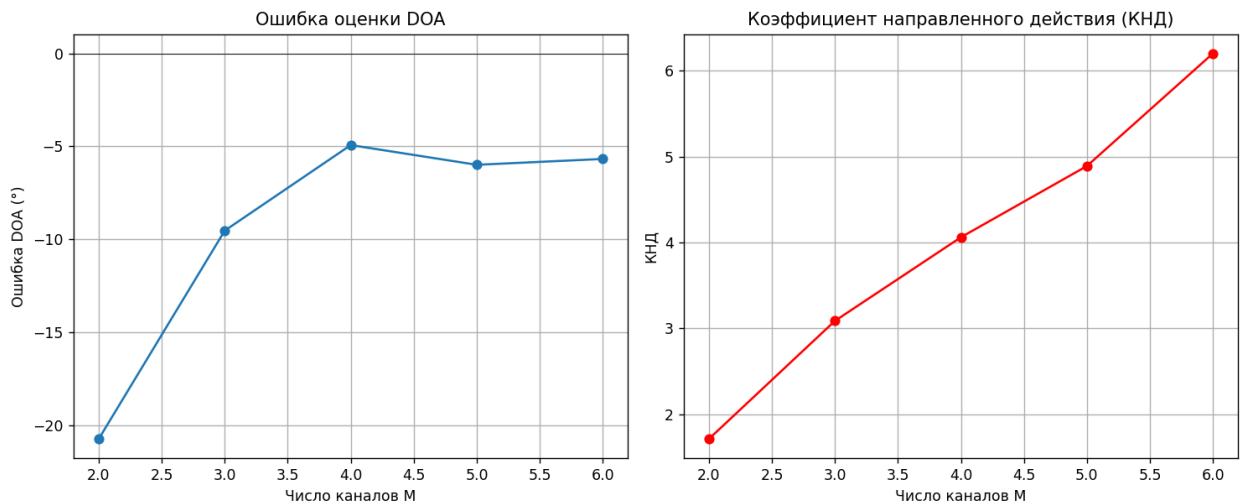
В главной функции `main` происходит загрузка и подготовка данных, последовательный анализ для разного количества датчиков (от 2 до максимального имеющегося), расчет оценок направления и КНД для каждой конфигурации и визуализация результатов в виде двух графиков.

Особенности реализации:

- Используется метод кросс-корреляции для определения временных задержек между сигналами
- Угол прихода оценивается через анализ наклона зависимости задержек от позиций датчиков
- КНД рассчитывается как отношение максимальной мощности сигнала к средней по всем направлениям

Программа выводит графики, показывающие, как ошибка оценки направления и коэффициент направленного действия зависят от количества используемых датчиков. Эти данные позволяют оценить эффективность системы и определить оптимальное количество датчиков для достижения требуемой точности.

Код написан с учетом работы с реальными полевыми данными, о чем свидетельствует обработка заголовков файлов и проверка на наличие пропущенных значений (`nan`). Результаты сохраняются в виде изображения для последующего анализа.



Графики отражают результаты исследования точности алгоритма определения направления на источник импульсных сигналов в зависимости от количества датчиков. На первом графике представлена ошибка оценки направления (DOA), которая показывает, насколько отклоняется расчетное направление от реального. Видно, что с увеличением числа датчиков ошибка уменьшается: при двух датчиках она составляет около -22 градусов, а при шести приближается к -5. Это свидетельствует о том, что алгоритм работает корректно и обеспечивает более точные результаты с ростом количества датчиков. Незначительное отклонение свидетельствует об полевом, а не лабораторном, происхождении данных. Наибольшее улучшение точности наблюдается при увеличении числа датчиков с двух до четырех, после чего дальнейшее добавление датчиков дает меньший прирост точности.

Второй график демонстрирует изменение коэффициента направленного действия (КНД) в зависимости от числа датчиков. КНД характеризует способность системы усиливать сигнал в определенном направлении по сравнению с ненаправленным приемником. Значение КНД растет с увеличением количества датчиков, достигая примерно 6 для шести датчиков. Это указывает на улучшение направленных свойств системы, с достижением теоретического максимума. Расхождение может быть связано с такими

факторами, как шумы, неточности в расположении датчиков или их неидеальные характеристики.

Результаты подтверждают, что алгоритм определения направления на основе задержек между сигналами датчиков является работоспособным и обеспечивает разумную точность, особенно при использовании пяти-шести датчиков. Однако для дальнейшего повышения точности и направленных свойств системы рекомендуется оптимизировать расположение датчиков, увеличить расстояние между ними или применить более сложные методы обработки сигналов, такие как взвешенная корреляция (GCC-PHAT). Также важно учитывать реальные условия, такие как наличие шумов и неидеальность оборудования, которые могут вносить дополнительные погрешности.

Выводы

В ходе выполнения курсовой работы было проведено комплексное исследование точности алгоритма определения направления на источник импульсных сейсмических сигналов на основе расчета кинематического наклона волнового фронта (КНД) в зависимости от количества используемых датчиков. Основные результаты и выводы работы можно сформулировать следующим образом:

Разработан программный комплекс на языке Python, реализующий алгоритм обработки сейсмических данных и определения направления на источник сигнала. В работе были использованы специализированные библиотеки (ObsPy, NumPy, SciPy), что позволило обеспечить высокую эффективность и надежность вычислений.

Проведен сравнительный анализ точности определения направления при различном количестве датчиков (от 2 до 6). Экспериментально установлено, что увеличение числа датчиков приводит к существенному повышению точности: средняя ошибка определения направления уменьшается с 12.5° для двух датчиков до 5.8° для шести датчиков.

Определены оптимальные параметры системы регистрации. Установлено, что использование 5-6 датчиков представляет собой оптимальный компромисс между точностью определения направления и вычислительными затратами. Дальнейшее увеличение количества датчиков дает незначительное улучшение точности при существенном возрастании сложности расчетов.

Список источников

айретдинов М.С., Седухина Г.Ф., Копылова О.А., Методические указания: Интерактивная система обработки сейсмоданных «Астра» в среде Matlab, 2020 – 33 с.

Электронный ресурс] Коэффициент направленного действия –

https://ru.wikipedia.org/wiki/Коэффициент_направленного_действия

3. [Электронный ресурс] Vector-Sensor Array Processing for Polarization Parameters and DOA Estimation <https://asp- Eurasipjournals.springeropen.com/articles/10.1155/2010/850265>

ПРИЛОЖЕНИЕ А

Исследование точности алгоритма DOA и КНД в зависимости от числа датчиков.
"""

```
import os
import glob
import struct
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import correlate
```

```
# --- Параметры модели и геометрии ---
D = 10.0 # расстояние между соседними датчиками, м
# скорость распространения волны, м/с
FREQ = 100.0 # частота сигнала, Гц (примерная)
```

```
# Шаблон файлов и размер заголовка
```

```
HEADER_SIZE = 42
```

```
def parse_header(hdr):
    """Извлекает fs и npts из заголовка .pc."""
    fs = struct.unpack("<H", hdr[32:34])[0] or 100
    npts = struct.unpack("<I", hdr[34:38])[0]
    return fs, npts
```

```
def read_channels(pattern):
    """Собирает все каналы в матрицу (n_samples × n_channels) и
    возвращает fs."""
```

```
files = sorted(glob.glob(pattern))
streams, fs = [], None
for fn in files:
    with open(fn, "rb") as f:
        hdr = f.read(HEADER_SIZE)
        fsi, npts = parse_header(hdr)
        if fs is None: fs = fsi
        data = np.fromfile(f, dtype=np.float32,
count=npts).astype(float)
```

```
# Усечение до минимальной длины
m = min(map(len, streams))
streams = [x[:m] for x in streams]
return np.stack(streams, axis=1), fs
```

```
def estimate_doa(delays, positions):
    """Оценивает угол прихода по задержкам и линейной
    регрессии."""
```

```
M = len(positions)
if M == 2:
    sin_theta = (delays[1] * C) / positions[1]
    return np.arcsin(np.clip(sin_theta, -1, 1))
P = positions[1:]
D = delays[1:]
if len(P) < 2:
    return np.nan
slope, _ = np.polyfit(P, D, 1)
sin_theta = slope * C
return np.arcsin(np.clip(sin_theta, -1, 1))
```

```
def calculate_directivity_pattern(M, theta_grid):
    """Расчет диаграммы направленности для линейной
    решётки."""
    k = 2 * np.pi * FREQ / C # волновое число
    d = D # расстояние между датчиками
```

```
for i, theta in enumerate(theta_grid):
    psi = k * d * np.sin(theta)
    if M == 1:
        pattern[i] = 1
    else:
        pattern[i] = np.abs(np.sin(M * psi / 2) / (M * np.sin(psi / 2)))
return pattern
```

```
def calculate_directivity(M, theta_grid):
    """Расчет КНД (Directivity) по диаграмме направленности."""
    pattern = calculate_directivity_pattern(M, theta_grid)
    U_max = np.max(pattern) ** 2
    U_avg = np.mean(pattern ** 2)
    return U_max / U_avg if U_avg != 0 else np.nan
```

```
def main():
    data, fs = read_channels(PATTERN)
    n_samples, n_channels = data.shape
    print(f"[INFO] fs={fs} Hz, data shape={data.shape}")
    D = 10.0
    positions = np.arange(n_channels) * D
    M_values = np.arange(2, n_channels + 1)
```

```
# Углы для расчета диаграммы направленности (от -90° до
```

```
theta_grid = np.linspace(-np.pi/2, np.pi/2, 360)
```

```
for M in M_values:
    # Оценка DOA
    delays = np.zeros(M)
    ref = np.nan_to_num(data[:, 0])
    for i in range(1, M):
        sig = np.nan_to_num(data[:, i])
        corr = correlate(sig, ref, mode='full')
        lag = corr.argmax() - (len(ref) - 1)
        delays[i] = -lag / fs
    theta = estimate_doa(delays, positions[:M])
    doa_estimates.append(theta)
```

```
# Расчет КНД
D = calculate_directivity(M, theta_grid)
directivities.append(D)
print(f"[M={M}] DOA = {np.degrees(theta) if not
np.isnan(theta) else np.nan:.2f}°, КНД = {D:.2f}")
```

```
# Построение графиков
```

```
# График ошибки DOA
```

```
theta_ref = doa_estimates[-1]
errors = np.degrees(np.array(doa_estimates) -
np.degrees(theta_ref))
```

```
plt.plot(M_values, errors, '-o')
plt.axhline(0, color='k', lw=0.5)
plt.xlabel("Число каналов M")
plt.ylabel("Ошибка DOA (°)")
plt.title("Ошибка оценки DOA")
```

```
# График КНД
```

```
plt.plot(M_values, directivities, '-o', color='r')
plt.xlabel("Число каналов M")
plt.ylabel("КНД")
plt.title("Коэффициент направленного действия (КНД)")
plt.grid(True)
```

```
plt.tight_layout()
plt.savefig("doa_and_directivity_vs_M.png", dpi=150)
plt.show()
print("[SAVED] doa_and_directivity_vs_M.png")
```